

Universidad Mariano Gálvez

Ingeniería en Sistemas de Información y Ciencias de la Computación

Programación III

Ing. David Marroquin



José Alfredo López Argueta

9989-23-16365

Guatemala, 2026

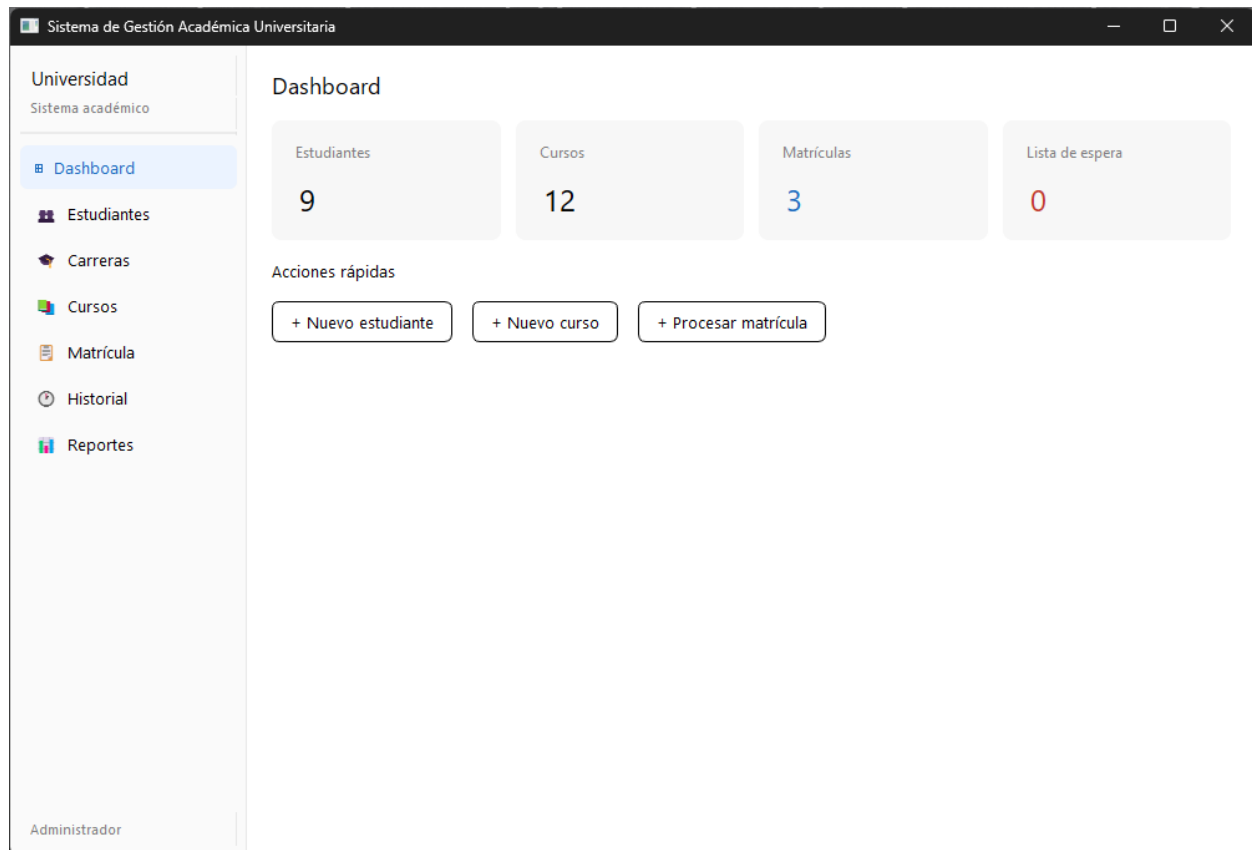
Sistema Inteligente de Gestión Académica Universitaria

1. Descripción General del Proyecto

El Sistema Inteligente de Gestión Académica Universitaria es una aplicación de escritorio desarrollada en C++ con interfaz gráfica Qt 6, diseñada para simular el funcionamiento de un registro académico universitario.

El sistema centraliza y automatiza las principales operaciones académicas de una institución educativa, permitiendo gestionar de forma eficiente a los estudiantes, los cursos disponibles, los procesos de matrícula, el historial académico y las relaciones de prerrequisitos entre cursos.

Una de sus características principales es que todas las estructuras de datos (pilas, colas, listas enlazadas, arboles BST, AVL, Árbol B, tabla hash y grafo dirigido) fueron implementadas desde cero en C++.



1.1 Problema que resuelve

Las universidades manejan grandes volúmenes de información académica diariamente. Sin un sistema adecuado, procesos como la verificación de prerrequisitos, el control de cupos, el cálculo de promedios y la generación de reportes se vuelven lentos y propensos a errores. Este sistema

proporciona una solución completa que automatiza dichos procesos garantizando consistencia e integridad en los datos.

1.2 Alcance del sistema

- Gestión de múltiples carreras universitarias
- Registro y administración de estudiantes
- Catálogo de cursos con control de cupos
- Proceso de matrícula con validación automática de prerrequisitos
- Lista de espera cuando el cupo está agotado
- Historial académico por estudiante
- Cálculo de promedio académico
- Generación de reportes exportables a CSV
- Simulación de asignación de aulas

2. Objetivos

2.1 Objetivo General

Desarrollar un sistema de escritorio en C++ con Qt 6 que gestione la información académica de una universidad utilizando estructuras de datos avanzadas implementadas desde cero, integrando persistencia en MySQL y una interfaz gráfica funcional.

2.2 Objetivos Específicos

1. Registrar estudiantes en el sistema académico con su información completa.
2. Administrar el catálogo de cursos de la universidad con control de cupos.
3. Simular el proceso de matrícula estudiantil mediante una cola de espera.
4. Mantener el historial académico de cada estudiante utilizando una pila.
5. Verificar automáticamente los prerrequisitos de los cursos mediante un grafo dirigido.
6. Calcular promedios académicos utilizando un árbol de expresiones matemáticas.
7. Permitir búsquedas rápidas de estudiantes utilizando tabla hash y árbol BST.
8. Guardar toda la información de forma persistente en una base de datos MySQL.
9. Generar reportes académicos exportables en formato CSV.

3. Tecnologías Utilizadas

Tecnología	Versión	Uso en el proyecto
C++	C++ 17	Lenguaje principal de desarrollo
Qt	Qt 6.5	Interfaz gráfica de usuario (GUI)
MySQL	8.0	Base de datos relacional para persistencia
MySQL ODBC Conector	9.7.0	Driver de conexión Qt-MySQL en Windows
CMake	3.19+	Sistema de construcción del proyecto
Git	2.46.0	Control de versiones del código fuente
Qt Creator	19.0.1	Entorno de desarrollo integrado (IDE)

4. Arquitectura del Sistema

El sistema está organizado en cuatro capas claramente diferenciadas, siguiendo el patrón de arquitectura en capas. Cada capa tiene una responsabilidad única y se comunica únicamente con la capa adyacente.

Capa	Carpeta	Responsabilidad
Presentación (UI)	ui/	Vistas Qt: ventanas, formularios, tablas
Lógica de Negocio	core/	Clases del dominio y controlador principal
Estructuras de Datos	structures/	Implementaciones propias de estructuras
Persistencia	database/ + MySQL	Conexión y operaciones CRUD en la BD

4.1 Descripción de capas

Capa de Presentación: contiene todas las vistas de la aplicación implementadas con Qt Widgets. Cada vista (Dashboard, Estudiantes, Cursos, Matricula, Historial, Reportes) es construida dinámicamente y mostrada en un QStackedWidget central.

Capa de Lógica de Negocio: el componente principal es Sistema Académico, implementado como Singleton, que actúa como coordinador central. Recibe peticiones de la UI, aplica las reglas de negocio (validación de prerrequisitos, control de cupos) y coordina las estructuras de datos con la base de datos.

Capa de Estructuras de Datos: biblioteca propia de estructuras implementadas como plantillas en archivos .h. Son independientes de la lógica de negocio y pueden reutilizarse en cualquier proyecto.

Capa de Persistencia: la clase ConexionDB implementa el patrón Singleton para mantener una única conexión activa durante toda la sesión. Se conecta a MySQL mediante el driver ODBC de Windows.

4.2 Patrón de diseño Singleton

Tanto ConexionDB como SistemaAcademico implementan el patrón Singleton, garantizando que solo exista una instancia de cada uno durante la ejecución del programa. Esto evita múltiples conexiones innecesarias a la base de datos y asegura coherencia en el estado del sistema.

5. Estructuras de Datos Implementadas

Todas las estructuras fueron implementadas desde cero utilizando templates de C++. Cada estructura reside en un archivo .h independiente dentro de la carpeta structures/.

Estructura	Archivo	Uso en el sistema
Pila (Stack)	Pila.h	Historial académico de cada estudiante (LIFO)
Cola (Queue)	Cola.h	Fila de espera en el proceso de matrícula (FIFO)
Lista Simple	ListaSimple.h	Registro general de todos los estudiantes
Lista Doble	ListaDoble.h	Catálogo de cursos con navegación bidireccional
BST	BST.h	Búsqueda de estudiantes por carnet
AVL	AVL.h	Clasificación de estudiantes por promedio balanceado
Árbol de Expresiones	ArbolExpresiones.h	Cálculo del promedio académico
Árbol B	ArbolB.h	Índice académico
Tabla Hash	TablaHash.h	Acceso instantáneo a estudiantes por carnet
Grafo Dirigido	Grafo.h	Prerrequisitos con DFS, BFS y detección de ciclos
Simulador de Memoria	SimuladorMemoria.h	Asignación de aulas: primer, mejor y peor ajuste

5.1 Justificación de cada estructura

Pila para historial académico: el historial se consulta en orden cronológico inverso (el curso más reciente primero), lo cual corresponde exactamente al comportamiento LIFO de una pila.

Cola para matricula: el proceso de atención a estudiantes sigue el principio FIFO, es decir, el primero en llegar es el primero en ser atendido. La cola modela naturalmente este comportamiento.

BST y Tabla Hash para búsqueda: el BST permite búsquedas en $O(\log n)$ con recorridos ordenados, mientras que la tabla hash permite búsqueda en $O(1)$ promedio. Ambas se usan en conjunto para maximizar el rendimiento.

AVL para promedios: al insertar estudiantes ordenados por promedio, un BST simple podría degenerarse. El AVL garantiza que el árbol permanezca balanceado con rotaciones automáticas.

Grafo dirigido para prerrequisitos: las relaciones entre cursos forman naturalmente un grafo dirigido acíclico (DAG). El algoritmo DFS permite recorrer todas las dependencias para validar si un estudiante cumple los prerrequisitos antes de matricularse.

6. Diseño de Base de Datos

La base de datos universidad_db fue diseñada en MySQL 8.0. Contiene 7 tablas que modelan todas las entidades del sistema y sus relaciones.

6.1 Tablas del sistema

Tabla	Descripción	Campos principales
carreras	Carreras universitarias disponibles	id (PK), nombre
estudiantes	Datos de cada estudiante	carnet (PK), nombre, edad, id_carrera (FK), fecha_ingreso
cursos	Catálogo de cursos	codigo (PK), nombre, creditos, cupo_maximo
curso_carrera	Relación curso-carrera (N:M)	codigo_curso (FK), id_carrera (FK)
prerrequisitos	Dependencias entre cursos	codigo_curso (FK), codigo_prereq (FK)
matriculas	Inscripciones de estudiantes	id (PK), carnet_estudiante (FK), codigo_curso (FK), ciclo, estado
historial_academico	Notas y estados por curso	id (PK), carnet (FK), codigo_curso (FK), nota, estado, ciclo, fecha
aulas	Aulas disponibles para asignación	id (PK), nombre, capacidad, ocupado, disponible

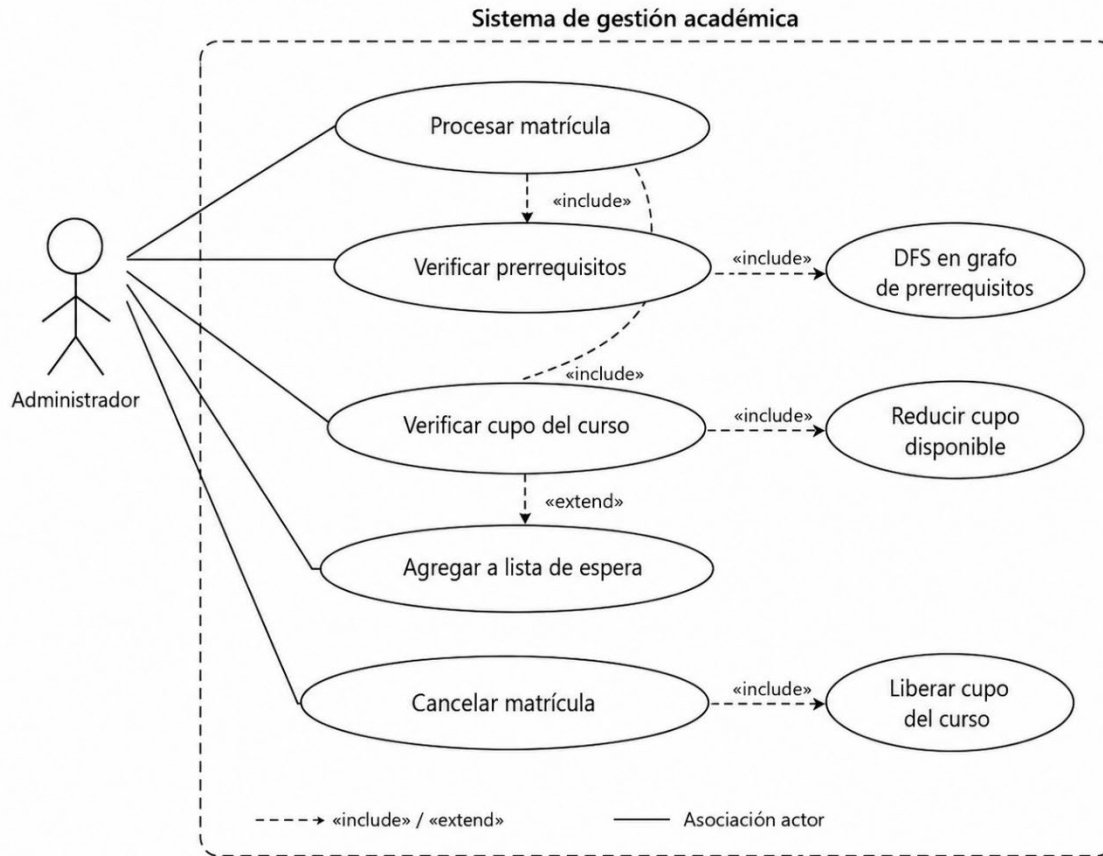
6.2 Relaciones entre tablas

- Un estudiante pertenece a una carrera (N a 1).
- Un curso puede pertenecer a varias carreras (N a M, resuelta con curso_carrera).
- Un estudiante puede tener muchas matriculas (1 a N).
- Una matrícula vincula un estudiante con uno o más cursos en un ciclo específico.
- La tabla prerrequisitos es una autorreferencia sobre cursos.
- El historial académico registra la nota final de cada curso.

7. Diagramas UML

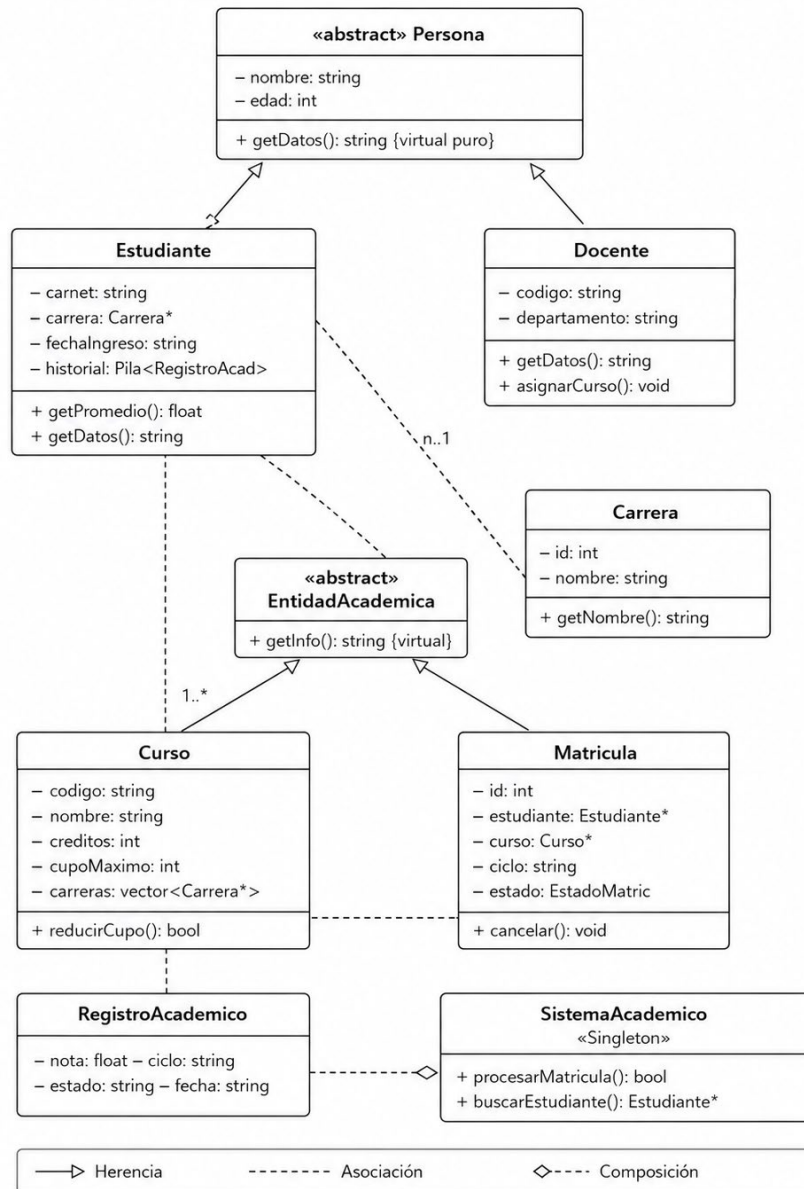
7.1 Diagrama de Casos de Uso

El sistema tiene un único actor: el Administrador.



7.2 Diagrama de Clases

La jerarquía de clases implementa herencia y polimorfismo. La clase abstracta Persona es la base de Estudiante y Docente. La clase abstracta Entidad Académica es la base de Curso y Matricula.

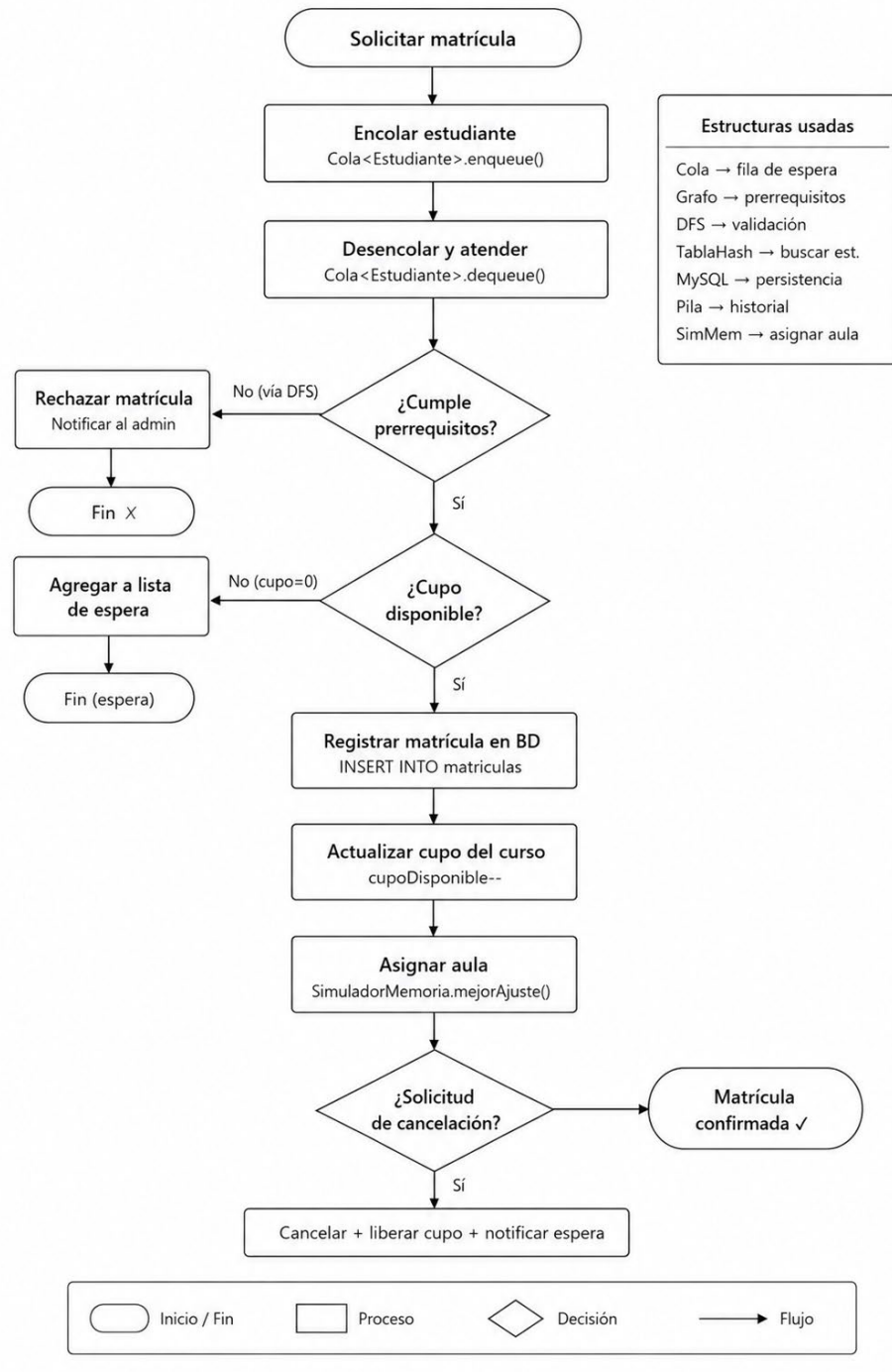


7.3 Flujo del Proceso de Matricula

El proceso de matrícula es el flujo más complejo del sistema. Involucra la cola de espera, la verificación de prerequisites mediante el grafo dirigido y el control de cupos.

1. El estudiante ingresa a la cola de matrícula.
2. El sistema desencola al estudiante y verifica sus prerequisites usando DFS en el grafo.
3. Si no cumple prerequisites, se rechaza la solicitud con mensaje de error.
4. Si cumple prerequisites, se verifica el cupo disponible del curso.
5. Si hay cupo, se registra la matricula como activa en la base de datos.
6. Si no hay cupo, la matricula queda en estado "en espera".

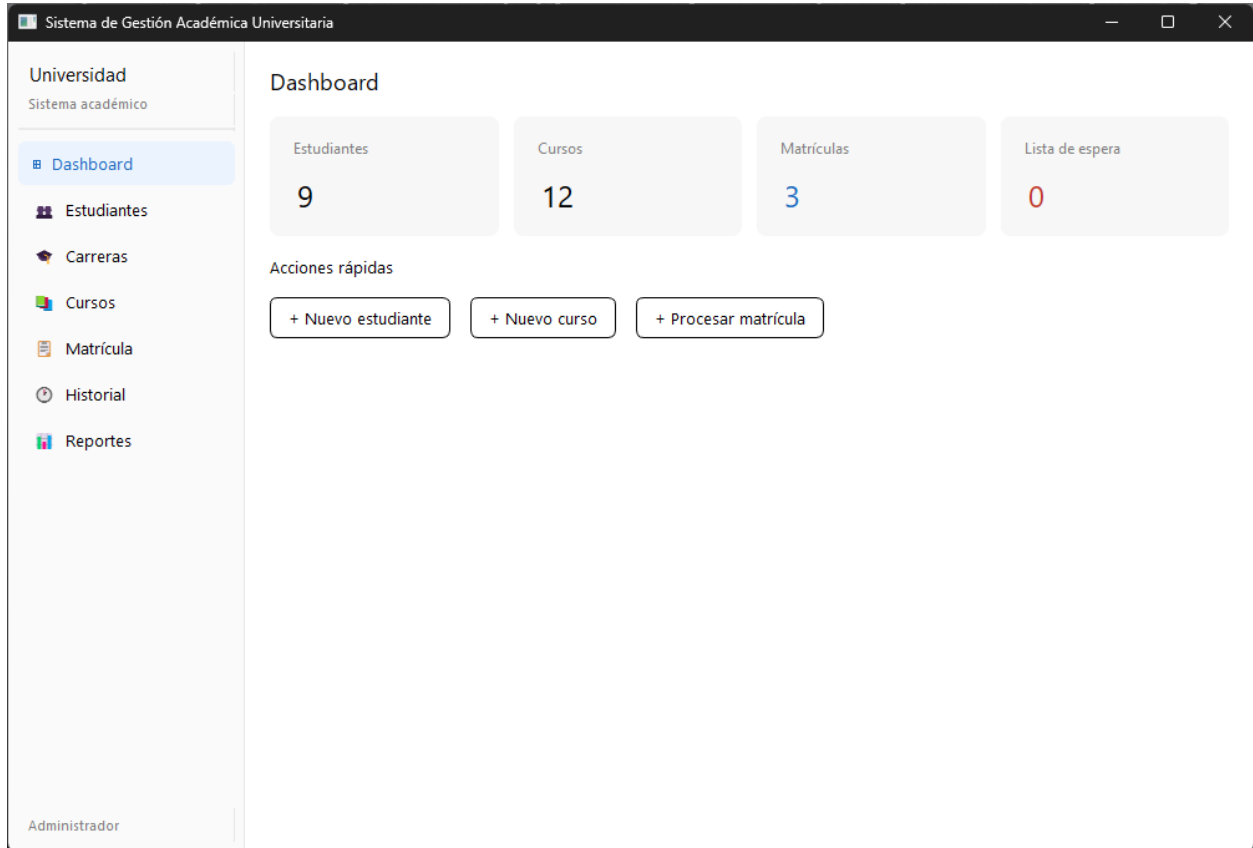
7. El cupo disponible se actualiza y se asigna un aula mediante mejor ajuste.



8. Funcionalidades del Sistema

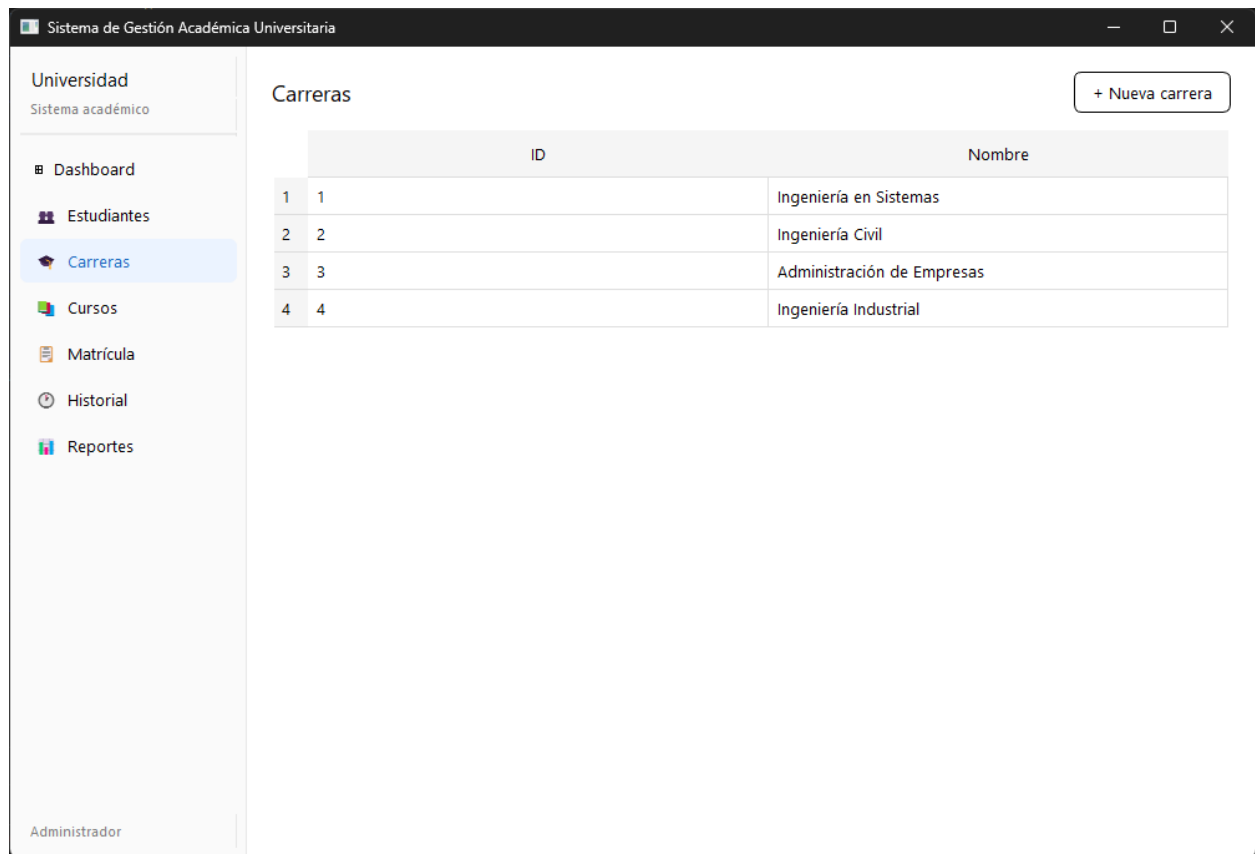
8.1 Dashboard

La pantalla principal muestra un resumen del estado del sistema con tarjetas de métricas: total de estudiantes registrados, total de cursos, matrículas activas y estudiantes en lista de espera. También proporciona accesos directos a las operaciones más frecuentes.



8.2 Gestión de Carreras

Permite registrar nuevas carreras universitarias y visualizar las existentes. Las carreras son la base del sistema ya que cada estudiante pertenece a una carrera específica.



8.3 Gestión de Estudiantes

Permite registrar nuevos estudiantes con carnet, nombre, edad, carrera y fecha de ingreso. Incluye búsqueda rápida por carnet utilizando la tabla hash, visualización del promedio académico calculado desde la base de datos y listado completo con todos los datos.

Sistema de Gestión Académica Universitaria

Universidad

Sistema académico

Dashboard

Estudiantes

Carreras

Cursos

Matrícula

Historial

Reportes

Administrador

Estudiantes

+ Nuevo estudiante

Buscar por carnet...

Buscar

Mostrar todos

	Carnet	Nombre	Carrera	Ingreso	Promedio	
1	2025001	Juan Perez	Ingeniería en ...	2025-01-10	—	Eliminar
2	2024005	Sofía Rodríguez	Administración de...	2022-01-12	—	Eliminar
3	2024004	José Martínez	Ingeniería en ...	2024-01-15	—	Eliminar
4	2024003	María López	Ingeniería Civil	2023-01-10	—	Eliminar
5	2024002	Carlos Pérez	Ingeniería en ...	2024-01-15	—	Eliminar
6	2024001	Ana García	Ingeniería en ...	2024-01-15	—	Eliminar
7	2023003	Diego Flores	Ingeniería en ...	2022-01-12	—	Eliminar
8	2023002	Andrea Torres	Ingeniería Civil	2023-01-10	—	Eliminar
9	2023001	Luis Hernández	Ingeniería en ...	2023-01-10	—	Eliminar

8.4 Gestión de Cursos

Permite registrar nuevos cursos con código, nombre, créditos y cupo máximo. Al registrar un curso se pueden especificar sus prerrequisitos directamente. El sistema verifica que los cursos prerrequisito existan antes de guardar la relación.

Sistema de Gestión Académica Universitaria

Universidad
Sistema académico

Dashboard

Estudiantes

Carreras

Cursos

Matrícula

Historial

Reportes

Administrador

Cursos

+ Nuevo curso

	Código	Nombre	Créditos	Cupo disponible
1	ADS101	Análisis de Sistemas	5	25/25
2	ALG101	Álgebra Lineal	3	30/30
3	BDD101	Bases de Datos I	3	25/25
4	BDD102	Bases de Datos II	4	25/25
5	FIS101	Física I	3	30/30
6	FIS102	Física II	3	30/30
7	MAT101	Matemática I	4	30/30
8	MAT102	Matemática II	4	30/30
9	PRG101	Programación I	4	25/25
10	PRG102	Programación II	4	25/25
11	PRG103	Estructuras de Datos	4	25/25
12	RED101	Redes de Computadoras	3	20/20

8.5 Proceso de Matrícula

El módulo de matrícula permite inscribir a un estudiante en múltiples cursos en un mismo ciclo. Para cada curso el sistema verifica automáticamente los prerrequisitos usando el grafo dirigido y el cupo disponible. El resultado muestra cuantos cursos fueron aceptados y cuales fueron rechazados con su motivo.

Sistema de Gestión Académica Universitaria

Universidad
Sistema académico

Dashboard

Estudiantes

Carreras

Cursos

Matrícula

Historial

Reportes

Administrador

Procesar matrícula

Carnet estudiante:ej. 2024001

Ciclo:ej. 2025-1

Cursos a matricular:

Código curso

Código del curso a agregar

+ Agregar curso

Procesar matrícula

8.6 Historial Académico y Notas

Permite consultar el historial académico completo de un estudiante por carnet. Muestra tanto los cursos matriculados actualmente como el historial de notas con su estado (aprobado/reprobado). También permite registrar nuevas notas para actualizar el promedio del estudiante.

Sistema de Gestión Académica Universitaria

Universidad

Sistema académico

Dashboard

Estudiantes

Carreras

Cursos

Matrícula

Historial

Reportes

Administrador

Historial académico y notas

Carnet del estudiante

Consultar

Código	Curso	Nota	Estado	Ciclo
--------	-------	------	--------	-------

Cursos matriculados

Código	Curso	Ciclo	Estado
--------	-------	-------	--------

Sistema de Gestión Académica Universitaria

Universidad

Sistema académico

Dashboard

Estudiantes

Carreras

Cursos

Matrícula

Historial

Reportes

Administrador

Historial académico y notas

2025001

Consultar

Estudiante: Juan Perez | Promedio: Sin notas

Código	Curso	Nota	Estado	Ciclo
--------	-------	------	--------	-------

Cursos matriculados

	Código	Curso	Ciclo	Estado
1	PRG101	Programación I	2025-1	Activo

Código curso:

ej. PRG101

Nota:

0 - 100

Ciclo:

ej. 2026-1

Fecha:

YYYY-MM-DD

Cancelar

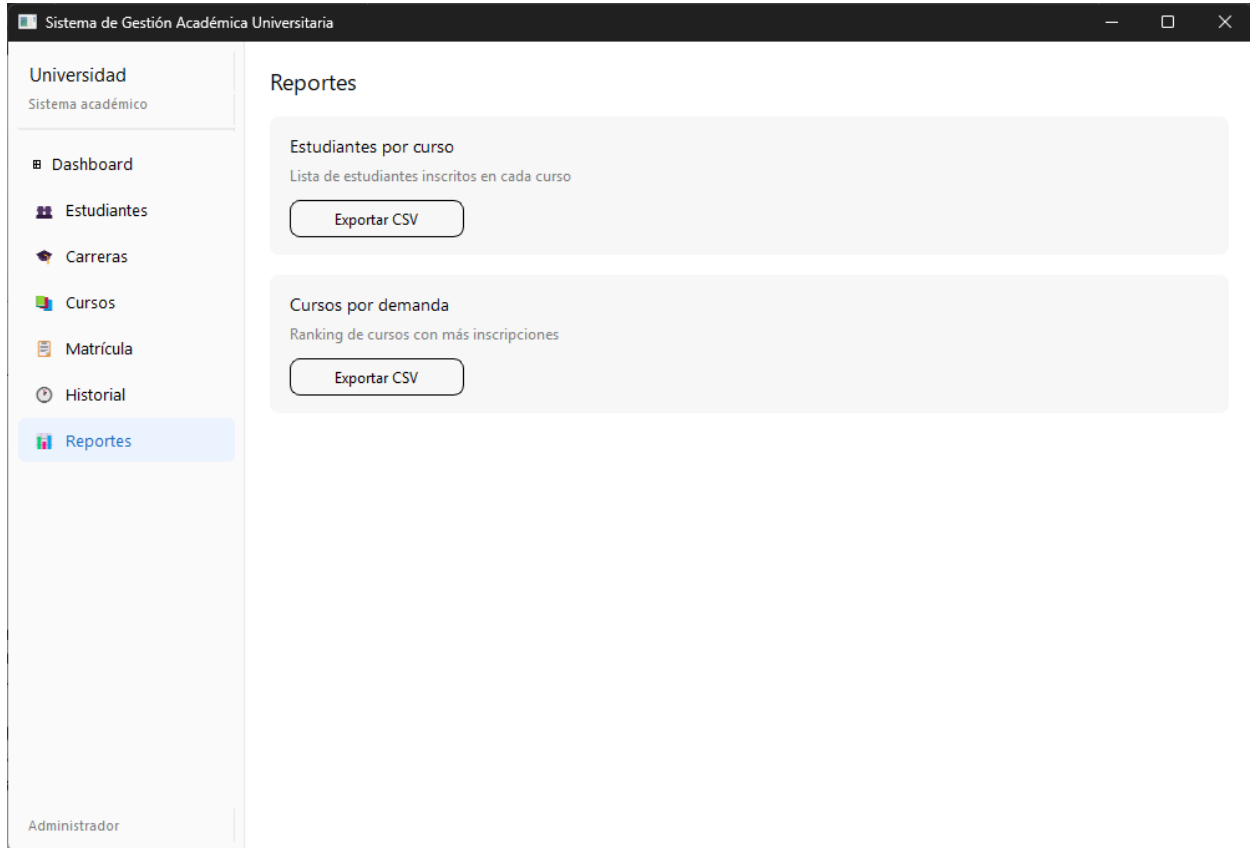
Guardar nota

+ Agregar nota

Eliminar último registro

8.7 Reportes

Genera dos reportes exportables en formato CSV: estudiantes inscritos por curso (muestra el listado de todos los estudiantes activos en cada curso) y cursos por demanda (ranking de cursos ordenado por número de inscripciones). Los archivos CSV se pueden abrir directamente en Excel.



9. Conclusiones

9.1 Logros del proyecto

- Se implementaron exitosamente las 11 estructuras de datos requeridas por el curso desde cero en C++.
- Se logro integrar correctamente C++ con MySQL.
- Se diseño e implemento una arquitectura modular en 4 capas que facilita el mantenimiento y la extensión del sistema.
- El sistema cubre todas las funcionalidades mínimas requeridas por el enunciado del proyecto y agrega funcionalidades adicionales como matricula múltiple por ciclo y simulación de asignación de aulas.
- Se utilizo Git para el control de versiones a lo largo de todo el desarrollo.

9.2 Aspectos más complejos

- La implementación del Árbol AVL fue el mayor reto técnico, ya que requiere manejar 4 tipos de rotaciones (simple izquierda, simple derecha, doble izquierda-derecha, doble derecha-izquierda) y actualizar las alturas correctamente en cada operación.
- La conexión de Qt 6 con MySQL en Windows requirió investigación adicional ya que Qt no incluye el driver QMYSQL precompilado, lo que me obligo a usar ODBC como puente.
- La validación de prerequisites mediante DFS en el grafo requirió cuidado especial para manejar correctamente los casos donde un curso tiene múltiples prerequisites encadenados.